



UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
Engenharia de Computação

ANA LAURA KOEFENDER FÜHR
LUIS GUSTAVO ERICH DUARTE

TRABALHO FINAL DE PROJETO E ANÁLISE DE ALGORITMOS I

Porto Alegre

2026

TRABALHO FINAL DE PROJETO E ANÁLISE DE ALGORITMOS I

Trabalho apresentado para a disciplina de Projeto e Análise de Algoritmos I, da Universidade Federal do Rio Grande do Sul, em 2026.

Prof. Dr. Bruno Iochins Grisci

Porto Alegre
2026

SUMÁRIO

1. INTRODUÇÃO.....	4
2. PROBLEMA 1.....	5
2.1 DESCRIÇÃO.....	5
2.2 ESTRATÉGIA.....	6
2.3 CORREÇÃO.....	7
2.4 COMPLEXIDADE.....	8
2.5 DISCUSSÃO.....	9
3. PROBLEMA 2.....	10
3.1 DESCRIÇÃO.....	10
3.2 ESTRATÉGIA.....	11
3.3 CORREÇÃO.....	12
3.4 COMPLEXIDADE.....	13
3.5 DISCUSSÃO.....	14
4. CONCLUSÃO.....	15
5. REFERÊNCIAS.....	16

1. INTRODUÇÃO

A busca por soluções computacionais que conciliam rapidez e eficácia é um dos principais objetivos da ciência da computação. Entre as diversas técnicas de projetos existentes, os algoritmos gulosos se destacam pelo seu baixo custo computacional .

Este trabalho analisa a aplicação de estratégias gulosas e de grafos em dois desafios específicos da plataforma Beecrowd, são estes: Problema 1: 1799 - O Rato no Labirinto e Problema 2: 1128 - Ir e Vir.

Por meio desses estudos de caso, este relatório visa explorar como propriedades fundamentais da teoria dos grafos se alinham a estratégias de otimização local, demonstrando a aplicação prática de algoritmos clássicos para solucionar problemas reais de logística e roteamento.

2. PROBLEMA 1 - O RATO NO LABIRINTO

2.1 DESCRIÇÃO

O problema proposto consiste na simulação e otimização do percurso de um rato geneticamente selecionado, chamado IBO, que deve navegar por um labirinto complexo para atingir um objetivo duplo. O cenário é modelado como um grafo não direcionado e sem pesos, onde as salas são representadas por vértices (identificados por letras ou palavras), e os caminhos disponíveis entre eles são representados por arestas (links).

A mecânica do desafio estabelece uma ordem obrigatória de ações que o personagem IBO deve tomar, IBO deve iniciar sua trajetória no ponto denominado "Entrada". A partir daí, seu primeiro objetivo é localizar e coletar uma recompensa (o queijo), que está posicionada em um vértice específico indicado pelo caractere "*". Somente após garantir o queijo, o rato está autorizado a prosseguir em direção ao ponto de evacuação, denominado "Saida".

O objetivo principal da tarefa é determinar o custo mínimo desse trajeto, expresso pelo número total de pontos que o personagem deve interagir ou passar ao longo de toda a jornada.

2.2 ESTRATÉGIA

Visto que, uma especificação crucial do enunciado dita que o labirinto permite a reutilização de caminhos – ou seja, se a rota mais eficiente exigir que o nosso personagem IBO transite por um mesmo ponto duas vezes, esse ponto deve ser contabilizado individualmente em cada passagem – a estratégia utilizada consiste em rodar o algoritmo de Dijkstra duas vezes, calculando de forma independente o menor caminho entre a Entrada e o queijo e somando-o ao menor caminho entre o queijo e a Saída.

2.3 CORREÇÃO

Como visto em aula, o algoritmo de Dijkstra utilizado com a fila de prioridade heap, garantidamente encontra o caminho de menor custo de um grafo valorado não-negativo. Portanto, ao atribuir o valor um para cada uma das arestas do grafo fornecido pelo usuário o algoritmo encontrará o caminho com o menor número de nodos mantendo a invariante no momento em que um vértice “ u ” é retirado da fila de prioridade e marcado como *processado*[u] = *true*, a distância armazenada em *dist*[u] já é o menor caminho definitivo da origem até o nó “ u ”. Logo, ao atingir a condição de parada, o algoritmo garante o retorno do menor valor de saltos possíveis da entrada até o queijo, e do queijo até a saída.

Por que a solução atende às exigências do problema?

O algoritmo atende às exigências do problema pois ele divide a busca em duas etapas, *dist1* e *dist2*, o que garante que se caso IBO passe por um mesmo ponto mais de uma vez, o número de vezes que IBO passou por esse mesmo ponto será contabilizado na distância total retornada pelo algoritmo.

2.4 COMPLEXIDADE

A análise da complexidade do algoritmo será feita em duas partes: a complexidade de tempo e a complexidade de espaço

2.4.1 Complexidade de Tempo

Para gerar o grafo o algoritmo roda exatamente m vezes (número de links), visto que as operações de inserir elementos no grafo tem custo de $O(1)$, podemos concluir que a complexidade total de gerar o grafo é $O(m)$

```
# Leitura da entrada:
n, m = (int(x) for x in input().split())
graph = {}
for _ in range(m):
    a, b = input().split()

    # Escreva aqui o código preenchendo o grafo
    if a not in graph:
        graph[a] = []
    if b not in graph:
        graph[b] = []

    #preenche o grafo coloca dist como 1
    graph[a].append((b, 1))
    graph[b].append((a, 1))
```

Figura 1 – Código que gera o grafo 1

Conforme visto em aula o algoritmo de Dijkstra tem complexidade de $O((m+n)\log n)$, porém o algoritmo roda duas vezes, uma encontrar o menor caminho da entrada até o queijo e outra do queijo até a saída, então podemos concluir que o custo é $2O((m+n)\log n)$.

Como as constantes podem ser ignoradas temos que a complexidade de tempo total do algoritmo desenvolvido é $O(m) + O((m+n)\log n)$, que é igual à $O((m+n)\log n)$.

2.4.2 Complexidade de Espaço

O grafo armazena n vértices (salas) e m arestas, logo a complexidade de memória do grafo é $O(n+m)$.

Na estrutura de Dijkstra temos as listas(dicionários), *processado*, *dist* e *parent* cada um com complexidade de $O(n)$, a fila de prioridade armazena as distâncias, portanto tem complexidade de $O(m)$, logo a complexidade total de espaço da estrutura Dijkstra é $3O(n) + O(m)$

```
5 def dijkstra(graph, source, fim):
6     # Inicialização
7     processado = {v : False for v in graph.keys()} # para marcar nós processados
8     dist = {v : float('inf') for v in graph.keys()}
9     parent = {v : None for v in graph.keys()}
10
11     dist[source] = 0 # Origem começa com distância zero
12
13     fila = [(0, source)]
14
15     while fila: # enquanto fila não for vazia
16         du, u = heapq.heappop(fila) # extrai nodo com menor distância na fila
17
18         if processado[u]:
19             continue
20         processado[u] = True
21
22         if u == fim:
23             return dist[fim]
24
25         if u in graph:
26             for v, wv in graph[u]: # para cada vizinho v de u
27                 novo_dv = du + wv # distância até u mais peso da aresta u-v
28                 if novo_dv < dist[v]: # se distância menor que a atual
29                     dist[v] = novo_dv
30                     parent[v] = u
31                     heapq.heappush(fila, (novo_dv, v)) # coloca v na fila com sua distância at
32
33     return float('inf') ##Caso não exista um caminho
```

Figura 2 – Código de Dijkstra

Por fim, a complexidade total de espaço do algoritmo é $O(m+n) + 3O(n) + O(m)$, que é igual a $O(m+n)$.

2.5 DISCUSSÃO

De modo geral, este foi um problema bem tranquilo de ser resolvido, as maiores dificuldades foram com a plataforma Beecrowd – pois ela esperava um tipo de saída específico, que era um número inteiro e o algoritmo retornava float, demorei um pouco para encontrar o problema por conta disso – e não com a lógica para desenvolver a solução em si.

3. PROBLEMA 2 - IR E VIR

3.1 DESCRIÇÃO

O segundo problema consiste em verificar a alcançabilidade total entre os pontos de uma cidade. Modelando a situação, as interseções representam os vértices de um grafo, enquanto as vias representam as arestas. Como as ruas podem possuir mão única ou dupla, deduz-se tratar de um grafo direcionado. A construção desse modelo exige a leitura de variáveis em triplas: as duas primeiras indicam os vértices conectados, e a terceira atua como um indicador de bidirecionalidade da via.

A partir da tradução dessas informações ilustrativas para uma perspectiva teórica, constata-se que o objetivo do problema é determinar se o grafo fornecido é Fortemente Conexo. Para solucionar essa questão, foi aplicado o algoritmo de Kosaraju-Shamir, previamente demonstrado em aula e disponível no repositório da disciplina, o qual atende perfeitamente a essa demanda estrutural.

3.2 ESTRATÉGIA

A estratégia utilizada foi de um algoritmo previamente desenvolvido para descobrir os componentes fortemente conexos, o Kosaraju Shamir, conforme a implementação do professor Lucas Alegre. Ele funciona baseado na busca por profundidade (DFS), e no pressuposto de que o dígrafo equivalente, mas com o sentido das arestas invertido, possui os mesmos componentes fortemente conexos que o original.

Diante disso, o algoritmo inicializa invertendo o grafo, e gerando uma organização topográfica desse grafo invertido, já que seu funcionamento só ocorre se a busca por profundidade for realizada em uma ordem específica. Em seguida, inicializamos um dicionário vazio dos vértices visitados e um contador para guardar a informação de qual componente cada um dos nodos pertence.

Posteriormente é executado uma busca por profundidade (DFS) e então um último for, que passa por cada vértice conforme a organização topográfica definida inicialmente, se ele encontrar um vértice novo, acabamos de descobrir outro componente fortemente conexo, e então chamamos a função DFS novamente, para registrar todos os “membros” desse novo componente.

Por último, o algoritmo testa o número de componentes fortemente conexos, representados pela variável c , e se obtermos apenas 1, ela retorna 1, representando algo verdadeiro, e caso contrário, ela retorna 0.

Além disso, foi implementada uma função para registrar os vértices e arestas em um dicionário, percorrendo cada um dos valores fornecidos pelo problema, e caso a variável indicativa de bilateralidade (p) for verdadeira, ingressamos em uma parte do código que adiciona a aresta saindo de ambos os vértices do grafo.

3.3 CORREÇÃO

Como visto nas aulas de Projeto e Análise de Algoritmos, durante o período responsável pelos algoritmos em grafos, a variação do algoritmo de Kosaraju-Shamir escolhida garante a identificação precisa dos Componentes Fortemente Conexos de um dígrafo. Ao realizar a primeira Busca em Profundidade (DFS) no grafo reverso para calcular a ordem topológica, o algoritmo constrói uma invariante matemática de que o vértice com o maior tempo de término na recursão pertencerá a um componente “fonte”.

Portanto, ao executar a segunda etapa de DFS seguindo a ordem dessa lista e marcando os vértices como visitados, o algoritmo garante que a exploração não extrapolará para componentes já processados, confinando a busca aos limites exatos de cada Componente Fortemente Conexo e rotulando-os corretamente conforme a atribuição “components[s] = c”. Logo ao finalizar essa travessia da lista principal, podemos confirmar o mapeamento de todas as regiões conectadas sem sobreposições.

Por que a solução atende às exigências do problema?

O algoritmo atende às exigências do problema “Ir e Vir” pois ele traduz a capacidade de partir de qualquer cruzamento e alcançar qualquer outro para a propriedade exata de um único Componente Fortemente Conexo.

Ao dividir a execução em duas etapas complementares, processando a ida no grafo original e a volta indiretamente mapeada pela ordem do grafo reverso, garantimos que, caso exista uma rua de mão única que impeça o retorno de um determinado bairro, essa falha dividirá o grafo em múltiplos componentes.

Dessa forma, o teste final contabiliza corretamente todo o mapa e torna-se uma prova definitiva de conectividade global, assegurando o retorno de que a cidade atende ou não ao requisito exigido.

3.4 COMPLEXIDADE

A análise da complexidade do algoritmo será feita em duas partes: a complexidade de tempo e a complexidade de espaço

3.4.1 Complexidade de Tempo

O algoritmo de Kosaraju-Shamir consiste em passos bem definidos baseados em Busca em Profundidade (DFS) para percorrer o grafo. Por não realizar verificações exaustivas e visitar cada vértice e aresta um número fixo de vezes, sua complexidade de tempo é linear.

Inicialmente inicializamos o grafo e recebemos os valores fornecidos pelo problema, colocando-os em um dicionário, que possui complexidade $O(n+m)$, sendo que n é o número de vértices, e m o número de arestas.

```
#parte de processamento dos grafos em dicionários
while True:
    n, m = (int(x) for x in input().split())

    if n == 0 and m == 0:
        break
    #inicializa os grafos normal e invertido

    graph = {i: [] for i in range(1, n + 1)}
    graph_rev = {i: [] for i in range(1, n + 1)}

    for _ in range(m):
        v, w, p = (int(x) for x in input().split())

        graph[v].append(w)
        graph_rev[w].append(v)

        # só adicionamos o caminho da volta se a rua for de mão dupla (p == 2)
        if p == 2:
            graph[w].append(v)
            graph_rev[v].append(w)
```

Figura 3 – Código que gera os dicionários

Logo após esse passo, foi chamada a função de Kosaraju-Sharir, desenvolvida pelo professor Lucas Alegre, e como foi destacado no próprio repositório, funciona com a complexidade de $O(n+m)$, dependendo apenas de outros passos como a ordenamento topológico (toposort) e inversão dos sentidos do dígrafo (reverse_graph), além do mapeamento de todos os

vértices como “não visitados” (init_visitados), que possuem uma complexidade igual ou inferior a isso (construção do grafo, inversão do grafo, ordenação topológica e busca de componentes, todos possuem complexidade $O(n+m)$).

```
def kosaraju_shamir(g):  
  
    """Algoritmo de Kosaraju-Shamir para encontrar Componentes Fortemente Conexas em um Dígrafo.  
    rev_g = reverse_graph(g)  
    order = toposort(rev_g)  
    resultado = 0  
  
    visited = init_visitados(g)  
    components = {}  
    c = 0  
  
    def dfs(g, s, visitados, components, c):  
        visitados[s] = True  
        components[s] = c  
        for v in g[s]:  
            if not visited[v]:  
                dfs(g, v, visited, components, c)  
  
    for i in order:  
        if not visited[i]:  
            dfs(g, i, visited, components, c)  
            c += 1  
  
    if (c == 1):  
        resultado = 1  
  
    return resultado
```

Figura 4 – Código da função de Kosaraju-Shamir

Diante disso, podemos concluir que a complexidade da solução encontrada para o segundo problema é $O(n+m)$, sendo n o número de vértices e m o número de arestas.

3.4.1 Complexidade de Espaço

Os dicionários que armazenam os vértices e arestas do grafo todos possuem complexidade espacial de $O(n+m)$, já as listas possuem complexidade $O(n)$. Logo a complexidade espacial do algoritmo é a maior de todos os processos, que é $O(n+m)$, com n sendo o número de vértices e m sendo o número de arestas.

3.5 DISCUSSÃO

A resolução deste problema foi relativamente fácil, pois pude utilizar os códigos do repositório da disciplina, combinados com as funções de criação de dicionários para grafos fornecidas para outros problemas do beecrowd. Além disso, tive que criar apenas o teste de sentido das vias, caso elas fossem ou não de mão dupla, e o retorno da função, que antes mostrava os Componentes Fortemente Conexos, e agora retorna apenas uma variável com valores “1” ou “0”.

4. CONCLUSÃO

Diante dos problemas apresentados, podemos concluir que os resultados obtidos foram satisfatórios, e estão de acordo com o esperado pela disciplina. Além disso, foi possível avaliar nossos conhecimentos como suficientes para a resolução de questões envolvendo Teoria dos Grafos, Análise de Complexidade e Corretude de Algoritmos.

5. REFERÊNCIAS

FEOFILOFF, Paulo. **Algoritmo de Kosaraju-Sharir para componentes fortes de grafos**. Algoritmos para Grafos (em linguagem C). São Paulo: IME-USP, [s.d.].

Disponível em:

https://www.ime.usp.br/~pf/algoritmos_para_grafos/aulas/kosaraju.html. Acesso em: 14 jun. 2026.